Implémentation des Widgets GTK4
Rapport Technique - Volume 2

Projet GTK Wrapper

March 8, 2026

Contents

1 Introduction **3**

2 Module `window.c` **4**
2.1 Fonction `apply_window_background` 4
2.2 Fonction `create_window` 5
2.3 APIs GTK utilisées 6

3 Module `button.c` **7**
3.1 Fonction `create_button` 7
3.2 Fonction `create_radio_button` 8
3.3 APIs GTK utilisées 8

4 Module `common.c` **9**
4.1 Fonction `apply_widget_style` 9
4.2 APIs GTK utilisées 10

5 Module `container.c` **11**
5.1 Fonction `create_grid` 11
5.2 Fonction `create_box` 11
5.3 Fonction `create_stack` 11
5.4 Helpers complémentaires 12
5.5 APIs GTK utilisées 13

6 Module `display.c` **14**
6.1 Fonction `create_progress_bar` 14
6.2 Fonction `create_spinner` 14
6.3 Fonction `create_label` 15
6.4 Fonction `create_image` 15
6.5 APIs GTK utilisées 16

7 Module `input.c` **17**
7.1 Fonction `create_entry` 17
7.2 Fonction `create_dropdown` 17
7.3 Fonction `create_spin_button` 18
7.4 APIs GTK utilisées 18

8 Module `toggle.c` **20**
8.1 Fonction `create_checkbox` 20
8.2 Fonction `create_switch_row` 20
8.3 APIs GTK utilisées 21

9 Module `date.c` **22**
9.1 Fonction `create_calendar` 22
9.2 APIs GTK utilisées 22

10 Module `menu.c` **24**
10.1 Fonction `register_item_action` 24
10.2 Fonction `append_item_recursive` 24
10.3 Fonctions `create_menubar` et `create_menu_panel` 25
10.4 APIs GTK utilisées 26

11 Module `dialog.c` **27**
11.1 Callback interne `on_alert_response` 27
11.2 Fonction `show_alert_dialog` 27
11.3 APIs GTK utilisées 29

12 Module `separator.c` **30**
12.1 Fonction `create_separator` 30
12.2 APIs GTK utilisées 30

1 Introduction

Ce rapport documente l'implémentation interne du wrapper GTK4. Contrairement au premier volume, centré sur les structures de configuration et l'usage de l'API, ce second document décrit la logique des fonctions, les appels GTK réellement utilisés et les choix techniques effectués dans chaque module.

Les extraits de code présentés ici reprennent l'implémentation réelle du projet. Les commentaires ont été retirés dans les listings afin de conserver une compilation LaTeX simple et robuste tout en gardant un style visuel identique au premier rapport.

2 Module `window.c`

Le module `window.c` construit les fenêtres principales de l'application. Il encapsule la création d'un `GtkApplicationWindow`, l'application des propriétés usuelles et la gestion d'un fond personnalisé par CSS.

2.1 Fonction `apply_window_background`

Cette fonction privée applique un style CSS ciblé sur une seule fenêtre. L'idée est de nommer le widget puis de générer dynamiquement une règle CSS qui peut définir soit une couleur de fond seule, soit une couleur combinée à une image.

```c
static void apply_window_background(const window_config *config, GtkWidget *window) {
    if (config == NULL || window == NULL || config->background_color == NULL || config->background_color[0] == '\0') {
        return;
    }

    const char *name = config->widget_name;
    if (name == NULL || name[0] == '\0') {
        name = "wrapper-window";
        gtk_widget_set_name(window, name);
    }

    GtkCssProvider *provider = gtk_css_provider_new();
    char *css;

    if (config->background_image_path != NULL && config->background_image_path[0] != '\0') {
        css = g_strdup_printf(
            "window#%s, window#%s > * {"
            "background-color: %s;"
            "background-image: url('%s');"
            "background-size: cover;"
            "background-position: center;"
            "}",
            name,
            name,
            config->background_color,
            config->background_image_path
        );
    } else {
        css = g_strdup_printf(
            "window#%s, window#%s > * { background-color: %s; }",
            name,
            name,
            config->background_color
        );
    }
```

```

    gtk_css_provider_load_from_string(provider, css);
    gtk_style_context_add_provider_for_display(
        gtk_widget_get_display(window),
        GTK_STYLE_PROVIDER(provider),
        GTK_STYLE_PROVIDER_PRIORITY_APPLICATION
    );

    g_free(css);
    g_object_unref(provider);
}
```

2.2 Fonction `create_window`

La fonction principale du module instancie la fenêtre, applique les propriétés déclarées dans `window_config`, délègue la personnalisation visuelle à `apply_window_background` puis applique le style commun partagé avec les autres widgets.

```
GtkWidget *create_window(GtkApplication *app, const window_config *
   config) {
    GtkWidget *window = gtk_application_window_new(app);
    if (config == NULL) {
        return window;
    }

    if (config->title) {
        gtk_window_set_title(GTK_WINDOW(window), config->title);
    }
    if (config->icon_name) {
        gtk_window_set_icon_name(GTK_WINDOW(window), config->
           icon_name);
    }
    if (config->widget_name) {
        gtk_widget_set_name(window, config->widget_name);
    }

    gtk_window_set_default_size(GTK_WINDOW(window), config->
       default_width, config->default_height);
    gtk_window_set_resizable(GTK_WINDOW(window), config->resizable)
       ;
    gtk_window_set_decorated(GTK_WINDOW(window), config->decorated)
       ;
    gtk_window_set_modal(GTK_WINDOW(window), config->modal);

    if (config->min_width > 0 || config->min_height > 0) {
        gtk_widget_set_size_request(window, config->min_width,
           config->min_height);
    }

    apply_window_background(config, window);
    apply_widget_style(window, &config->style);
```

```

    if (config->maximized) {
        gtk_window_maximize(GTK_WINDOW(window));
    }
    if (config->present_on_create) {
        gtk_window_present(GTK_WINDOW(window));
    }

    return window;
}
```

2.3 APIs GTK utilisées

API	Rôle
`gtk_application_window_new()`	Crée une fenêtre associée à une instance de `GtkApplication`.
`gtk_widget_set_name()`	Attribue un identifiant CSS au widget pour cibler une seule fenêtre.
`gtk_css_provider_new()`	Prépare un fournisseur de style CSS chargé dynamiquement.
`gtk_style_context_add_provider_for_display()`	Enregistre le style CSS au niveau de l'affichage.
`gtk_window_set_default_size()`	Définit la taille initiale de la fenêtre.
`gtk_window_present()`	Affiche la fenêtre et demande le focus.

3 Module button.c

Le module button.c regroupe la création des boutons standards et des boutons radio. Il ajoute au widget GTK de base une logique de thème, d'icône et de raccordement de callbacks.

3.1 Fonction create_button

La fonction create_button traite plusieurs cas d'usage avec une seule interface : bouton texte, bouton avec mnémonique, bouton avec icône, ou combinaison texte plus icône. La variante de thème est appliquée séparément via une fonction interne dédiée.

```c
GtkWidget *create_button(const button_config *config) {
    GtkWidget *button;
    const char *label;

    if (config == NULL) {
        return gtk_button_new();
    }

    label = config->label ? config->label : "";
    button = config->use_underline ? gtk_button_new_with_mnemonic(label) : gtk_button_new_with_label(label);
    gtk_button_set_has_frame(GTK_BUTTON(button), config->has_frame);
    gtk_button_set_can_shrink(GTK_BUTTON(button), config->can_shrink);

    if (config->icon_name && config->icon_name[0] != '\0') {
        GtkWidget *content = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 6);
        GtkWidget *icon = gtk_image_new_from_icon_name(config->icon_name);
        GtkWidget *text = config->use_underline ? gtk_label_new_with_mnemonic(label) : gtk_label_new(label);
        if (config->icon_size > 0) {
            gtk_image_set_pixel_size(GTK_IMAGE(icon), config->icon_size);
        }
        gtk_box_append(GTK_BOX(content), icon);
        gtk_box_append(GTK_BOX(content), text);
        gtk_button_set_child(GTK_BUTTON(button), content);
    }

    apply_theme_variant(button, config->theme_variant);
    apply_widget_style(button, &config->style);

    if (config->on_clicked) {
        g_signal_connect(button, "clicked", G_CALLBACK(config->on_clicked), config->user_data);
```

```
    }

    return button;
}
```

3.2 Fonction `create_radio_button`

Le wrapper radio repose sur `GtkCheckButton`, comme le recommande GTK4. Le groupement exclusif est obtenu en reliant un bouton au précédent du groupe.

```
GtkWidget *create_radio_button(const radio_button_config *config) {
    GtkWidget *button = gtk_check_button_new_with_label(config &&
       config->label ? config->label : "");

    if (config == NULL) {
        return button;
    }

    if (config->group_with != NULL) {
        gtk_check_button_set_group(GTK_CHECK_BUTTON(button),
           GTK_CHECK_BUTTON(config->group_with));
    }
    gtk_check_button_set_active(GTK_CHECK_BUTTON(button), config->
       is_active);
    apply_widget_style(button, &config->style);

    if (config->on_toggled) {
        g_signal_connect(button, "toggled", G_CALLBACK(config->
           on_toggled), config->user_data);
    }

    return button;
}
```

3.3 APIs GTK utilisées

API	Rôle
`gtk_button_new_with_label()`	Crée un bouton texte simple.
`gtk_button_new_with_mnemonic()`	Active les raccourcis clavier fondés sur un underscore dans le label.
`gtk_button_set_child()`	Remplace le contenu interne du bouton par un conteneur personnalisé.
`gtk_image_new_from_icon_name()`	Construit l'icône à partir du thème courant.
`gtk_check_button_set_group()`	Regroupe plusieurs boutons en sélection exclusive.
`g_signal_connect()`	Connecte les signaux `clicked` et `toggled` aux callbacks fournis.

4 Module `common.c`

Le module `common.c` fournit la couche de style commune à l'ensemble du wrapper. Il évite la duplication de code en regroupant dans une seule fonction l'application des classes CSS, des marges, de l'alignement, de l'expansion et des métadonnées de confort comme l'infobulle.

4.1 Fonction `apply_widget_style`

La fonction `apply_widget_style` prend un widget GTK déjà créé et lui applique les attributs décrits dans `widget_style_config`. L'implémentation distingue bien les valeurs toujours appliquées, comme les marges, et les valeurs conditionnelles, pilotées par des indicateurs `set_*`.

```c
void apply_widget_style(GtkWidget *widget, const widget_style_config *style) {
    if (widget == NULL || style == NULL) {
        return;
    }

    if (style->css_class && style->css_class[0] != '\0') {
        gtk_widget_add_css_class(widget, style->css_class);
    }
    if (style->css_classes != NULL) {
        for (int i = 0; style->css_classes[i] != NULL; i++) {
            gtk_widget_add_css_class(widget, style->css_classes[i]);
        }
    }

    if (style->width_request != 0 || style->height_request != 0) {
        gtk_widget_set_size_request(widget, style->width_request, style->height_request);
    }

    gtk_widget_set_margin_top(widget, style->margin_top);
    gtk_widget_set_margin_bottom(widget, style->margin_bottom);
    gtk_widget_set_margin_start(widget, style->margin_start);
    gtk_widget_set_margin_end(widget, style->margin_end);

    if (style->set_halign) {
        gtk_widget_set_halign(widget, style->halign);
    }
    if (style->set_valign) {
        gtk_widget_set_valign(widget, style->valign);
    }
    if (style->set_hexpand) {
        gtk_widget_set_hexpand(widget, style->hexpand);
    }
    if (style->set_vexpand) {
        gtk_widget_set_vexpand(widget, style->vexpand);
```

```
    }
    if (style->set_sensitive) {
        gtk_widget_set_sensitive(widget, style->sensitive);
    }
    if (style->set_visible) {
        gtk_widget_set_visible(widget, style->visible);
    }

    if (style->tooltip && style->tooltip[0] != '\0') {
        gtk_widget_set_tooltip_text(widget, style->tooltip);
    }
}
```

4.2 APIs GTK utilisées

API	Rôle
`gtk_widget_add_css_class()`	Ajoute une ou plusieurs classes CSS widget.
`gtk_widget_set_size_request()`	Fixe une taille minimale souhaitée.
`gtk_widget_set_margin_top()` et variantes	Applique les marges externes du widg
`gtk_widget_set_halign()` et `gtk_widget_set_valign()`	Positionne le widget dans l'espace disp
`gtk_widget_set_hexpand()` et `gtk_widget_set_vexpand()`	Indique si le widget peut occuper l' libre du conteneur.
`gtk_widget_set_tooltip_text()`	Définit le texte affiché au survol.

5 Module `container.c`

Le module `container.c` rassemble les conteneurs de mise en page du wrapper. Il couvre la grille, la boîte linéaire, la pile de pages et deux helpers complémentaires pour les interfaces multi-vues.

5.1 Fonction `create_grid`

La fonction `create_grid` instancie un `GtkGrid` puis applique l'espacement entre lignes et colonnes ainsi que les options d'homogénéité.

```
GtkWidget *create_grid(const grid_config *config) {
    GtkWidget *grid = gtk_grid_new();
    if (config == NULL) {
        return grid;
    }

    gtk_grid_set_column_spacing(GTK_GRID(grid), config->
       column_spacing);
    gtk_grid_set_row_spacing(GTK_GRID(grid), config->row_spacing);
    gtk_grid_set_column_homogeneous(GTK_GRID(grid), config->
       homogeneous_columns);
    gtk_grid_set_row_homogeneous(GTK_GRID(grid), config->
       homogeneous_rows);
    apply_widget_style(grid, &config->style);
    return grid;
}
```

5.2 Fonction `create_box`

```
GtkWidget *create_box(const box_config *config) {
    GtkWidget *box = gtk_box_new(
        config ? config->orientation : GTK_ORIENTATION_VERTICAL,
        config ? config->spacing : 0
    );
    if (config == NULL) {
        return box;
    }

    gtk_box_set_homogeneous(GTK_BOX(box), config->homogeneous);
    apply_widget_style(box, &config->style);
    return box;
}
```

5.3 Fonction `create_stack`

```
GtkWidget *create_stack(const stack_config *config) {
    GtkWidget *stack = gtk_stack_new();
```

```
    if (config == NULL) {
        return stack;
    }

    gtk_stack_set_transition_type(GTK_STACK(stack), config->transition);
    gtk_stack_set_transition_duration(GTK_STACK(stack), config->duration_ms);
    gtk_stack_set_vhomogeneous(GTK_STACK(stack), config->vhomogeneous);
    gtk_stack_set_hhomogeneous(GTK_STACK(stack), config->hhomogeneous);
    apply_widget_style(stack, &config->style);
    return stack;
}
```

5.4 Helpers complémentaires

Le module ajoute aussi deux helpers pratiques : `create_stack_sidebar`, qui relie une barre latérale à une pile existante, et `stack_add_page`, qui ajoute une page puis renseigne son titre par propriété GTK.

```
GtkWidget *create_stack_sidebar(GtkWidget *stack, const widget_style_config *style) {
    GtkWidget *sidebar = gtk_stack_sidebar_new();
    if (stack != NULL) {
        gtk_stack_sidebar_set_stack(GTK_STACK_SIDEBAR(sidebar), GTK_STACK(stack));
    }
    apply_widget_style(sidebar, style);
    return sidebar;
}

void stack_add_page(GtkWidget *stack, GtkWidget *child, const char *name, const char *title) {
    if (stack == NULL || child == NULL || name == NULL || title == NULL) {
        return;
    }

    gtk_stack_add_named(GTK_STACK(stack), child, name);
    g_object_set(gtk_stack_get_page(GTK_STACK(stack), child), "title", title, NULL);
}
```

5.5 APIs GTK utilisées

API	Rôle
`gtk_grid_new()`	Crée un conteneur en grille.
`gtk_box_new()`	Crée un conteneur linéaire horizontal ou vertical.
`gtk_stack_new()`	Crée un conteneur à pages multiples.
`gtk_stack_sidebar_new()`	Crée le widget de navigation latérale lié à une pile.
`gtk_stack_add_named()`	Ajoute une page identifiée dans la pile.
`g_object_set()`	Attribue le titre de page utilisé par les widgets de navigation.

6 Module `display.c`

Le module `display.c` prend en charge les widgets d'affichage pur : barre de progression, spinner, label et image. Son rôle est d'exposer des constructeurs simples au-dessus des widgets GTK standards.

6.1 Fonction `create_progress_bar`

```c
GtkWidget *create_progress_bar(const progress_bar_config *config) {
    GtkWidget *progress = gtk_progress_bar_new();

    if (config == NULL) {
        return progress;
    }

    gtk_progress_bar_set_fraction(GTK_PROGRESS_BAR(progress),
       config->fraction);
    gtk_progress_bar_set_show_text(GTK_PROGRESS_BAR(progress),
       config->show_text);
    gtk_progress_bar_set_inverted(GTK_PROGRESS_BAR(progress),
       config->inverted);
    gtk_orientable_set_orientation(GTK_ORIENTABLE(progress), config
       ->orientation);
    if (config->text != NULL) {
        gtk_progress_bar_set_text(GTK_PROGRESS_BAR(progress),
           config->text);
    }
    if (config->pulse_once) {
        gtk_progress_bar_pulse(GTK_PROGRESS_BAR(progress));
    }
    apply_widget_style(progress, &config->style);

    return progress;
}
```

6.2 Fonction `create_spinner`

```c
GtkWidget *create_spinner(const spinner_config *config) {
    GtkWidget *spinner = gtk_spinner_new();

    if (config != NULL) {
        if (config->size > 0) {
            gtk_widget_set_size_request(spinner, config->size,
               config->size);
        }
        if (config->spinning) {
            gtk_spinner_start(GTK_SPINNER(spinner));
        }
        apply_widget_style(spinner, &config->style);
```

```
    }

    return spinner;
}
```

6.3 Fonction create_label

```
GtkWidget *create_label(const label_config *config) {
    GtkWidget *label = gtk_label_new(config && config->text ?
       config->text : "");

    if (config == NULL) {
        return label;
    }

    gtk_label_set_selectable(GTK_LABEL(label), config->selectable);
    gtk_label_set_wrap(GTK_LABEL(label), config->wrap);
    gtk_label_set_ellipsize(GTK_LABEL(label), config->ellipsize);
    if (config->max_width_chars > 0) {
        gtk_label_set_max_width_chars(GTK_LABEL(label), config->
           max_width_chars);
    }
    apply_widget_style(label, &config->style);

    return label;
}
```

6.4 Fonction create_image

```
GtkWidget *create_image(const image_config *config) {
    GtkWidget *widget;

    if (config == NULL) {
        return gtk_image_new();
    }

    if (config->file_path != NULL && config->file_path[0] != '\0')
       {
        GtkWidget *pic = gtk_picture_new_for_filename(config->
           file_path);
        gtk_picture_set_can_shrink(GTK_PICTURE(pic), TRUE);
        gtk_picture_set_content_fit(
            GTK_PICTURE(pic),
            config->keep_aspect ? GTK_CONTENT_FIT_CONTAIN :
               GTK_CONTENT_FIT_FILL
        );
        widget = pic;
    } else if (config->icon_name != NULL && config->icon_name[0] !=
        '\0') {
```

```
        GtkWidget *img = gtk_image_new_from_icon_name(config->
            icon_name);
        if (config->pixel_size > 0) {
            gtk_image_set_pixel_size(GTK_IMAGE(img), config->
                pixel_size);
        }
        widget = img;
    } else {
        widget = gtk_image_new();
    }

    apply_widget_style(widget, &config->style);
    return widget;
}
```

6.5 APIs GTK utilisées

API	Rôle
`gtk_progress_bar_set_fraction()`	Positionne l'avancement sur une valeur normalisée.
`gtk_progress_bar_pulse()`	Déclenche un avancement indéterminé ponctuel.
`gtk_spinner_start()`	Lance l'animation du spinner.
`gtk_label_set_ellipsize()`	Contrôle la coupure des textes trop longs.
`gtk_picture_new_for_filename()`	Crée un widget d'image basé sur un fichier.
`gtk_image_new_from_icon_name()`	Crée une icône issue du thème GTK.

7 Module input.c

Le module input.c regroupe les composants de saisie. Il encapsule trois widgets principaux : l'entrée de texte, la liste déroulante moderne et le bouton numérique à incréments.

7.1 Fonction create_entry

Cette fonction prépare un GtkEntry puis applique le placeholder, le texte initial, le type de saisie attendu et les éventuelles contraintes de visibilité ou de longueur.

```
GtkWidget *create_entry(const entry_config *config) {
    GtkWidget *entry = gtk_entry_new();

    if (config == NULL) {
        return entry;
    }

    if (config->placeholder) {
        gtk_entry_set_placeholder_text(GTK_ENTRY(entry), config->
           placeholder);
    }
    if (config->default_text) {
        gtk_editable_set_text(GTK_EDITABLE(entry), config->
           default_text);
    }
    gtk_entry_set_input_purpose(GTK_ENTRY(entry),
       parse_input_purpose(config->purpose_hint));
    gtk_entry_set_visibility(GTK_ENTRY(entry), !config->password);
    gtk_editable_set_editable(GTK_EDITABLE(entry), config->editable
       );
    if (config->max_length > 0) {
        gtk_entry_set_max_length(GTK_ENTRY(entry), config->
           max_length);
    }
    apply_widget_style(entry, &config->style);

    if (config->on_changed) {
        g_signal_connect(entry, "changed", G_CALLBACK(config->
           on_changed), config->user_data);
    }

    return entry;
}
```

7.2 Fonction create_dropdown

```
GtkWidget *create_dropdown(const dropdown_config *config) {
    GtkWidget *dropdown;

    if (config == NULL || config->options == NULL) {
```

```
        return gtk_drop_down_new(NULL, NULL);
    }

    dropdown = gtk_drop_down_new_from_strings(config->options);
    gtk_drop_down_set_selected(GTK_DROP_DOWN(dropdown), config->
       selected_index);
    gtk_drop_down_set_enable_search(GTK_DROP_DOWN(dropdown), config
       ->enable_search);
    apply_widget_style(dropdown, &config->style);

    if (config->on_selected) {
        g_signal_connect(dropdown, "notify::selected", G_CALLBACK(
           config->on_selected), config->user_data);
    }

    return dropdown;
}
```

7.3 Fonction create_spin_button

```
GtkWidget *create_spin_button(const spin_button_config *config) {
    GtkAdjustment *adj;
    GtkWidget *spin;

    if (config == NULL) {
        adj = gtk_adjustment_new(0.0, 0.0, 100.0, 1.0, 5.0, 0.0);
        return gtk_spin_button_new(adj, 1.0, 0);
    }

    adj = gtk_adjustment_new(config->value, config->min_value,
       config->max_value, config->step, config->page, 0.0);
    spin = gtk_spin_button_new(adj, config->step, config->digits);
    gtk_spin_button_set_numeric(GTK_SPIN_BUTTON(spin), config->
       numeric_only);
    gtk_spin_button_set_wrap(GTK_SPIN_BUTTON(spin), config->wrap);
    apply_widget_style(spin, &config->style);

    if (config->on_value_changed) {
        g_signal_connect(spin, "value-changed", G_CALLBACK(config->
           on_value_changed), config->user_data);
    }

    return spin;
}
```

7.4 APIs GTK utilisées

API	Rôle
`gtk_entry_set_placeholder_text()`	Définit le texte d'indication d'un champ vide.
`gtk_entry_set_input_purpose()`	Informe GTK du type de saisie attendu.
`gtk_drop_down_new_from_strings()`	Construit une liste de choix depuis un tableau de chaînes.
`gtk_drop_down_set_enable_search()`	Active la recherche intégrée du widget déroulant.
`gtk_adjustment_new()`	Définit l'intervalle numérique et les pas du contrôle.
`gtk_spin_button_new()`	Crée le widget de saisie numérique associé à un ajustement.

8 Module `toggle.c`

Le module `toggle.c` couvre les composants binaires du wrapper : case à cocher et ligne composée avec interrupteur. Ces deux widgets reposent sur des contrôles GTK simples, mais le wrapper leur ajoute une configuration déclarative homogène.

8.1 Fonction `create_checkbox`

```
GtkWidget *create_checkbox(const checkbox_config *config) {
    GtkWidget *checkbox = gtk_check_button_new_with_label(config &&
        config->label ? config->label : "");

    if (config == NULL) {
        return checkbox;
    }

    gtk_check_button_set_active(GTK_CHECK_BUTTON(checkbox), config
       ->active);
    gtk_check_button_set_inconsistent(GTK_CHECK_BUTTON(checkbox),
       config->inconsistent);
    apply_widget_style(checkbox, &config->style);

    if (config->on_toggled != NULL) {
        g_signal_connect(checkbox, "toggled", G_CALLBACK(config->
           on_toggled), config->user_data);
    }

    return checkbox;
}
```

8.2 Fonction `create_switch_row`

Cette seconde fonction construit un composant composite : un label à gauche, un `GtkSwitch` à droite, puis un éventuel callback sur la propriété `active`.

```
GtkWidget *create_switch_row(const switch_config *config, GtkWidget
    **out_switch) {
    GtkWidget *row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    GtkWidget *label = gtk_label_new(config && config->label ?
       config->label : "");
    GtkWidget *sw = gtk_switch_new();

    gtk_widget_set_hexpand(label, TRUE);
    gtk_widget_set_halign(label, GTK_ALIGN_START);
    gtk_widget_set_halign(sw, GTK_ALIGN_END);
    gtk_box_append(GTK_BOX(row), label);
    gtk_box_append(GTK_BOX(row), sw);

    if (config != NULL) {
        gtk_switch_set_active(GTK_SWITCH(sw), config->active);
```

```
        gtk_switch_set_state(GTK_SWITCH(sw), config->state);
        apply_widget_style(row, &config->style);

        if (config->on_active_changed != NULL) {
            g_signal_connect(sw, "notify::active", G_CALLBACK(
               config->on_active_changed), config->user_data);
        }
    }

    if (out_switch != NULL) {
        *out_switch = sw;
    }

    return row;
}
```

8.3 APIs GTK utilisées

API	Rôle
`gtk_check_button_new_with_label()`	Crée la case à cocher avec son libellé.
`gtk_check_button_set_inconsistent()`	Active l'état visuel intermédiaire.
`gtk_switch_new()`	Crée l'interrupteur binaire GTK4.
`gtk_switch_set_active()`	Positionne l'état logique initial du switch.
`gtk_box_append()`	Assemble le label et l'interrupteur dans une même ligne.
`g_signal_connect()`	Relie les changements d'état aux callbacks utilisateur.

9 Module `date.c`

Le module `date.c` encapsule le widget calendrier de GTK4. Il prend en charge la date initiale, l'affichage des noms de jours, les numéros de semaine et la remontée du signal de sélection.

9.1 Fonction `create_calendar`

```c
GtkWidget *create_calendar(const calendar_config *config) {
    GtkWidget *calendar = gtk_calendar_new();

    if (config == NULL) {
        return calendar;
    }

    if (config->selected_date != NULL) {
#if GTK_CHECK_VERSION(4, 20, 0)
        gtk_calendar_set_date(GTK_CALENDAR(calendar), config->selected_date);
#else
        gtk_calendar_select_day(GTK_CALENDAR(calendar), config->selected_date);
#endif
    }
    gtk_calendar_set_show_day_names(GTK_CALENDAR(calendar), config->show_day_names);
    gtk_calendar_set_show_week_numbers(GTK_CALENDAR(calendar), config->show_week_numbers);
    gtk_calendar_set_show_heading(GTK_CALENDAR(calendar), !config->no_month_change);
    apply_widget_style(calendar, &config->style);

    if (config->on_day_selected != NULL) {
        g_signal_connect(calendar, "day-selected", G_CALLBACK(config->on_day_selected), config->user_data);
    }

    return calendar;
}
```

9.2 APIs GTK utilisées

API	Rôle
`gtk_calendar_new()`	Crée l'instance du calendrier.
`gtk_calendar_set_date()`	Positionne la date courante dans les versions récentes de GTK.
`gtk_calendar_select_day()`	Assure la compatibilité avec les versions plus anciennes.
`gtk_calendar_set_show_week_numbers()`	Active l'affichage du numéro de semaine.
`gtk_calendar_set_show_heading()`	Contrôle l'en-tête et la navigation mensuelle.
`g_signal_connect()`	Raccorde le signal de sélection de jour au callback.

10 Module menu.c

Le module menu.c implémente l'architecture moderne des menus GTK4. Les éléments visibles sont décrits via GMenuModel, tandis que la logique applicative est fournie sous forme d'actions enregistrées dans un GActionMap.

10.1 Fonction register_item_action

Cette fonction privée crée une action GTK pour une entrée de menu active, connecte le callback utilisateur puis enregistre l'action dans la map de l'application.

```
static void register_item_action(const menubar_config *config,
                                 const menu_item_config *item,
                                 const char *action_name) {
    if (config == NULL || config->action_map == NULL || item ==
       NULL || item->on_activate == NULL || action_name == NULL) {
        return;
    }

    GSimpleAction *action = g_simple_action_new(action_name, NULL);
    g_signal_connect(action, "activate", G_CALLBACK(item->
       on_activate), item->user_data);
    g_action_map_add_action(config->action_map, G_ACTION(action));
    g_object_unref(action);
}
```

10.2 Fonction append_item_recursive

La construction du menu est récursive. Le code traite d'abord les sous-menus et les sections, puis les actions simples en leur attribuant un nom unique.

```
static void append_item_recursive(GMenu *menu,
                                  const menu_item_config *item,
                                  const menubar_config *config) {
    if (item == NULL || item->enabled == false) {
        return;
    }

    if (item->kind == MENU_ITEM_SUBMENU && item->children != NULL
       && item->child_count > 0) {
        GMenu *submenu = g_menu_new();
        for (size_t i = 0; i < item->child_count; i++) {
            append_item_recursive(submenu, &item->children[i],
               config);
        }
        g_menu_append_submenu(menu, item->label ? item->label : "",
            G_MENU_MODEL(submenu));
        g_object_unref(submenu);
        return;
    }

```

```
    if (item->kind == MENU_ITEM_SECTION && item->children != NULL
       && item->child_count > 0) {
        GMenu *section = g_menu_new();
        for (size_t i = 0; i < item->child_count; i++) {
            append_item_recursive(section, &item->children[i],
               config);
        }
        g_menu_append_section(menu, item->label, G_MENU_MODEL(
           section));
        g_object_unref(section);
        return;
    }

    char *action_name = create_action_name();
    register_item_action(config, item, action_name);

    char *detailed_action = g_strdup_printf("app.%s", action_name);
    g_menu_append(menu, item->label ? item->label : "",
       detailed_action);

    g_free(detailed_action);
    g_free(action_name);
}
```

10.3 Fonctions `create_menubar` et `create_menu_panel`

Les fonctions publiques consomment un modèle construit par `build_menu_model`. La première fabrique une barre de menus, la seconde un panneau de menu réutilisable dans un popover.

```
GtkWidget *create_menubar(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *bar = gtk_popover_menu_bar_new_from_model(model);
    if (config != NULL) {
        apply_widget_style(bar, &config->style);
    }
    g_object_unref(model);
    return bar;
}

GtkWidget *create_menu_panel(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *panel = gtk_popover_menu_new_from_model(model);

    if (config != NULL) {
        gtk_orientable_set_orientation(
            GTK_ORIENTABLE(panel),
            config->layout == MENU_LAYOUT_VERTICAL ?
               GTK_ORIENTATION_VERTICAL :
               GTK_ORIENTATION_HORIZONTAL
        );
```

```
        gtk_popover_set_has_arrow(GTK_POPOVER(panel), config->
            show_arrow_indicators);
        apply_widget_style(panel, &config->style);
    }

    g_object_unref(model);
    return panel;
}
```

10.4 APIs GTK utilisées

API	Rôle
`g_simple_action_new()`	Crée l'action logique associée à un item de menu.
`g_action_map_add_action()`	Enregistre l'action dans la carte fournie par l'application.
`g_menu_append_submenu()`	Ajoute un sous-menu hiérarchique au modèle.
`g_menu_append_section()`	Crée une section logique de menu.
`gtk_popover_menu_bar_new_from_model()`	Instancie la barre de menus GTK4 à partir du modèle.
`gtk_popover_menu_new_from_model()`	Produit un panneau de menu pouvant être affiché dans un popover.

11 Module `dialog.c`

Le module `dialog.c` implémente les boîtes d'alerte asynchrones. Il s'appuie sur `GtkAlertDialog`, qui ne bloque pas l'exécution du programme et impose donc un traitement différé de la réponse utilisateur.

11.1 Callback interne `on_alert_response`

Cette fonction récupère le résultat asynchrone, transforme une éventuelle erreur en indice `-1`, puis transmet la réponse au callback utilisateur avant de libérer les ressources temporaires.

```c
static void on_alert_response(GObject *source_object, GAsyncResult
    *result, gpointer user_data) {
    (void)source_object;
    alert_async_context *ctx = user_data;
    GError *error = NULL;
    int index = gtk_alert_dialog_choose_finish(ctx->dialog, result,
        &error);
    if (error != NULL) {
        g_error_free(error);
        index = -1;
    }
    if (ctx->on_response != NULL) {
        ctx->on_response(index, ctx->user_data);
    }
    g_object_unref(ctx->dialog);
    g_free(ctx);
}
```

11.2 Fonction `show_alert_dialog`

La fonction publique configure le message, les détails, la modalité et la liste des boutons. Elle déduit aussi le bouton par défaut et le bouton d'annulation à partir de la configuration avant d'afficher le dialogue en mode synchrone visuel mais asynchrone du point de vue du code.

```c
void show_alert_dialog(const alert_dialog_config *config) {
    GtkAlertDialog *dialog;
    const char **buttons;

    if (config == NULL || config->message == NULL) {
        return;
    }

    dialog = gtk_alert_dialog_new("%s", config->message);
    if (config->detail != NULL) {
        gtk_alert_dialog_set_detail(dialog, config->detail);
    }
    gtk_alert_dialog_set_modal(dialog, config->modal);

    if (config->button_count > 0 && config->buttons != NULL) {
```

```c
        buttons = g_new0(const char *, config->button_count + 1);
        for (size_t i = 0; i < config->button_count; i++) {
            buttons[i] = config->buttons[i].label;
        }
        gtk_alert_dialog_set_buttons(dialog, buttons);
        g_free(buttons);
    }

    int default_button = config->default_button;
    int cancel_button = config->cancel_button;

    if (config->buttons != NULL && config->button_count > 0) {
        for (size_t i = 0; i < config->button_count; i++) {
            const char *appearance = config->buttons[i].appearance;
            if (appearance == NULL) {
                continue;
            }
            if (default_button < 0 && strcmp(appearance, "suggested") == 0) {
                default_button = (int)i;
            }
            if (cancel_button < 0 && strcmp(appearance, "cancel") == 0) {
                cancel_button = (int)i;
            }
        }
    }

    if (default_button >= 0) {
        gtk_alert_dialog_set_default_button(dialog, default_button);
    }
    if (cancel_button >= 0) {
        gtk_alert_dialog_set_cancel_button(dialog, cancel_button);
    }

    if (config->on_response != NULL) {
        alert_async_context *ctx = g_new0(alert_async_context, 1);
        ctx->dialog = g_object_ref(dialog);
        ctx->on_response = config->on_response;
        ctx->user_data = config->user_data;
        gtk_alert_dialog_choose(dialog, config->parent, NULL, on_alert_response, ctx);
    } else {
        gtk_alert_dialog_show(dialog, config->parent);
    }

    g_object_unref(dialog);
}
```

11.3 APIs GTK utilisées

API	**Rôle**
`gtk_alert_dialog_new()`	Crée le dialogue d'alerte GTK4.
`gtk_alert_dialog_set_buttons()`	Définit l'ensemble des boutons proposés à l'utilisateur.
`gtk_alert_dialog_set_default_button()`	Choisit le bouton activé par défaut.
`gtk_alert_dialog_choose()`	Lance l'affichage asynchrone avec callback de fin.
`gtk_alert_dialog_choose_finish()`	Récupère l'indice du bouton choisi dans le résultat asynchrone.
`gtk_alert_dialog_show()`	Affiche le dialogue sans traitement de réponse personnalisé.

12 Module separator.c

Le module separator.c est volontairement minimal. Il fournit un simple constructeur pour les séparateurs visuels, tout en restant cohérent avec le reste du wrapper grâce à widget_style_config.

12.1 Fonction create_separator

```
GtkWidget *create_separator(const separator_config *config) {
    GtkWidget *separator = gtk_separator_new(config ? config->
       orientation : GTK_ORIENTATION_HORIZONTAL);
    if (config != NULL) {
        apply_widget_style(separator, &config->style);
    }
    return separator;
}
```

12.2 APIs GTK utilisées

API	Rôle
gtk_separator_new()	Crée un séparateur horizontal ou vertical selon l'orientation demandée.